Advanced Apache Administration with Perl

by Casey West

Overview

Dynamic Apache Configuration

Configuring Apache dynamically is the only way to stay sane when the installations grow exponentially. Your time can never scale, but your software can.

Using Perl

Perl is a flexible, easy to use, widely installed programming language to help get the job done.

Using Templates

Templates offer one way of configuring Apache. They're useful if you don't have mod_perl available, or don't want it. It also adds a layer in the process that can be useful if you have to roll back a configuration, or for the paranoid, to double check your work.

Using mod_perl

mod_perl allows for an even more dynamic configuration approach that can provide very valuable hooks into Apache. Further, your configuration can happen last minute, that's good for many situations.

On Lots and Lots of Computers

For Lots and Lots of Customers

As already stated, we're talking about impressive scale. It's not impossible to grow to thousands of computers or customers. Always be prepared.

Audience

Anyone that Hates Doing it Twice

Large Apache Installations

I'm specifically targeting the lazy. That is, the people who are willing to work extra hard to be lazy for the long hall. It may be OK to do it twice, but certainly not two hundred times.

Why?

User-capable Configuration

We all have users, some of us have more than others. Users are unreasonable. They want to be hands-on, savvy, and in the know. In short, they want to be in charge. That's OK, by keeping them happy we keep our jobs.

Centralized Configuration

It's difficult to ssh into twenty servers and update Apache on all of them to proxy the image server to its new internal IP address. It's much easier to update a database and run a remote command to SIGHUP Apache.

Staying Lazy

Finally, the lazy rule the world. Why? Because we have the time.

Common Things to Automate

Virtual Hosts

Single biggest concern on the minds of large installation.

Rewrite Rules

Second biggest concern.

Basically Anything

Since I'm making that up, I should tell the truth. People want to automate just about anything. Seriously, how could we guess what that might be?

Solving the Virtual Hosts Problem

Assumptions

In solving the most common problem I've come in contact with, this example will make some basic assumptions. These assumptions may not work for you, but that won't matter much. The basic ideals are easily transferable to a host (no pun intended) of applications.

Relational Database

For centralized configuration.

Perl

Our universal hammer.

mod_perl

Our Swiss-Army Hammer.

You Have a Couple Thousand Servers to Configure

Just keep this in the back of your head as we walk through.

Typical VirtualHost

IP Based

```
<VirtualHost 209.295.191.9>
  ServerName caseywest.com
```

```
    ServerAlias  www.caseywest.com  geeknest.com


    DocumentRoot  /home/cwest/htdocs
    ScriptAlias   /cgi-bin/  "/home/cwest/cgi-bin/"


    ErrorLog    logs/caseywest.com-error_log
    CustomLog   logs/caseywest.com-access_log  combined
</VirualHost>
```

Here is a basic Virtual Host configuration. It's IP based and doesn't do too many custom things. This will be our "one of a thousand" example configuration from here on in.

Problems

I Wrote It by Hand

I had to write this configuration by hand.

Remember, We have Lots of These

Yuck, that means I had to write all 999 of the other virtual hosts by hand, too.

VirtualHost Template

Template

```
<VirtualHost [% ip_address %]>
  ServerName [% host_name %]
   ServerAlias www.[% host_name %] [% other_names.join(' ') %]


   DocumentRoot /home/[% user %]/htdocs
   ScriptAlias /cgi-bin/ "/home/[% user %]/cgi-bin/"


   ErrorLog    logs/[% host_name %]-error_log
   CustomLog   logs/[% host_name %]-access_log combined
</VirtualHost>
```

This looks better. Here I'm using syntax from the popular Perl Module, Template Toolkit. You can easily see the variables in our configuration that could apply to all 1000 of our virtual hosts.

Variables

ip_address

The IP address this virtual host binds to.

host_name

The hostname for this virtual host.

other_names

Aliases for this virtual host.

user

The user account that owns this virtual host.

Data Sources

Many Good Options

There are lots of good formats to keep your virtual host information in. Some are better than others, and better is subjective, so pick what works for you.

Flat Files

XML, YAML, CSV, or worse. It might be good enough for you. If it is, don't be ashamed to use it.

DBM Files

These can be tricky, but some of us are Old Skool hackers and really appreciate DBM files.

Relational Databases

My favorite solution. MySQL, Postgres, Oracle, SQLite, Sybase. All of these are good options.

Magic

SOAP, XML-RPC, LDAP, Custom Hacks. Again, whatever works for you.

Using MySQL

Schema

```
create table users (
  id        int      not null auto_increment,
  username varchar not null
);


create table servers (
  id        int      not null auto_increment,
  hostname varchar not null
);


create table virtalhosts (
  id          int      not null auto_increment,
  user        int      not null,
  server      int      not null,
  hostname    varchar not null,
  ip_address varchar not null
);
```

```
create table serveraliases (
  id           int      not null auto_increment,
  virtualhost int      not null,
  hostname     varchar not null
);
```

Here is a simple schema that encapsulates the data for the problem we're trying to solve. Four tables that represent various bits of information in a multi-user, multi-server environment. There could be a lot more information here, as we'll see later, but this is a fine start.

Digression 1: Mapping SQL to Objects

Using Class::DBI::mysql

```
package Hosting;
```

This declaration needs to be here to make Perl happy. It also helps in grouping a bunch of classes together under one general name. In this case the Perl module Hosting.pm contains all the classes for dealing with our hosting data, in the database.

```
package Hosting::Object;
  use base qw[Class::DBI::mysql];
  __PACKAGE__->set_db( 'Main', 'dsn', 'username', 'password' );
```

This base class sets up a database connection, and subclasses Class::DBI::mysql itself. This is a handy way of saving a few lines of code in the classes that model the data.

```
package Hosting::User;
  use base qw[Hosting::Object];
  __PACKAGE__->set_up_table( 'users' );
  __PACKAGE__->has_many( virtualhosts => 'Hosting::VirtualHost' );
  __PACKAGE__->has_a( server => 'Hosting::Server' );
```

Interface to the user table. Users have a few relationships: many virtual hosts, and a server.

```
package Hosting::Server;
  use base qw[Hosting::Object];
  __PACKAGE__->set_up_table( 'servers' );
  __PACKAGE__->has_many( users => 'Hosting::User' );
```

Interface to the server table. Servers just have one relationship: many users.

```
package Hosting::VirtualHost;
  use base qw[Hosting::Object];
  __PACKAGE__->set_up_table( 'virtualhosts' );
  __PACKAGE__->has_many( serveraliases => 'Hosting::ServerAlias' );
  __PACKAGE__->has_a( user => 'Hosting::User' );
  __PACKAGE__->add_constructor( by_hostname => 'hostname = ?' );
  sub aliases { map $_->hostname, $_[0]->serveraliases }
```

Interface to the virtualhost table. Virtual hosts have two relationships: many server aliases, and a user they belong to. Since we'll more often know the hostname of a virtual host as opposed to its id, a constructor is added to allow us to use that data more effectively. Also, since we will often just want a list of server aliases a convenience method is added that maps server aliases' hostnames and returns the list.

```
package Hosting::ServerAlias;
   use base qw[Hosting::Object];
    __PACKAGE__->set_up_table( 'serveraliases' );


1;
```

Finally, the interface to the serveralias table. This one is easy, no relationships or special extras required.

Digression 1: Mapping SQL to Objects

Usage

```
my $vhost = Hosting::VirtualHost->by_hostname( "caseywest.com" );
```

We retrieve a virtual host object using our previously specified constructor.

```
printf "ServerName %s\n", $vhost->hostname;
```

Accessing instance methods work like you might expect.

```
printf "DocumentRoot /home/%s/htdocs\n",
   $vhost->user->username;
```

In turn, by specifying relationships the user() instance method returns a Hosting::User object.

```
my @aliases = $vhost->aliases;
```

Finally, as explained, getting a list of server aliases is simple.

Digression 2: Templating Exposed

Using Template Toolkit

```
#!/usr/bin/perl
# make_vhost.pl
use warnings;
use strict;


use Hosting;
use Template;


die "Usage: $0 vhostname\n" unless @ARGV;
my ($vhostname) = @ARGV;


my $vhost = Hosting::VirtualHost->by_hostname( $vhostname );
if ( $vhost ) {
   my $tt = Template->new;
```

```
    $tt->process( \*DATA, { vhost => $vhost } );
} else {
    die "No host named $vhostname!\n";
}
```

This program will build a VirtualHost section of the httpd.conf file for a given virtual host name. If the database has information pertaining to the hostname specified we use the Template Toolkit to parse the template and print out the results. Otherwise we die with the appropriate error. In this case, the template is stored at the end of the program in the DATA section.

Digression 2: Templating Exposed

Using Template Toolkit Template

```
__END__
<VirtualHost [% vhost.ip_address %]>
    ServerName [% vhost.hostname %]
    ServerAlias www.[% vhost.hostname %] [% vhost.aliases.join(' ') %]

    DocumentRoot /home/[% vhost.user.username %]/htdocs
    ScriptAlias /cgi-bin/ "/home/[% vhost.user.username %]/cgi-bin/"

    ErrorLog    logs/[% vhost.hostname %]-error_log
    CustomLog   logs/[% vhost.hostname %]-access_log combined
</VirtualHost>
```

Here is our example template. It looks dangerously close to my first virtual host template example. In this case we've passed the object directly to the Template Toolkit as a template variable. Since we can use it directly, we do so.

Now What?

Full Server Configuration

For some of us the next logical step would be to template everything. As we look over a typical httpd.conf file it's easy to spot all sorts of information that would make sense to centralize and configure more dynamically.

Templating High Level Config

Template

```
[% root = "/usr/local/apache" %]
ServerType standalone
ServerRoot [% root %]
PidFile [% root %]/logs/httpd.pid
Timeout 300
KeepAlive On
MaxKeepAliveRequests 100
KeepAliveTimeout 15
MinSpareServers 10
```

```
MaxSpareServers 25
StartServers 15
MaxClients 80
MaxRequestsPerChild 50
ExtendedStatus On
```

Apache processes often come under scrutiny when trying to eek out the best overall performance for a web site. If you are administrating an Apache cluster it's nice to change values in one place to ensure that no one configuration is left out.

Templating Basic Server Config

Template

```
Port 80
User nobody
Group nobody
ServerAdmin casey@geeknest.com
ServerName [% server.hostname %]
ServerSignature On
```

Dynamic configuration of these values may be considered less important by most of us. Even so, we can see that the ServerName is already useful to template if we're working with multiple servers.

Templating Document Handling

Template

```
DocumentRoot [% root %]/htdocs
<Directory />
  AllowOverride None
</Directory>
<Directory "[% root %]/htdocs">
  Order deny,allow
  Deny from all
</Directory>
```

There is so much information in these sorts of directives. Just like virtual hosts, life is often made easier when applying templates to them.

Templating Logging

Template

```
ErrorLog [% root %]/logs/error_log
LogLevel warn
LogFormat "%h %l %u %t \"%r\" %>s %b \"%{Referer}i\" \"%{User-Agent}i\"" combined
LogFormat "%h %l %u %t \"%r\" %>s %b" common
LogFormat "%{Referer}i -> %U" referer
LogFormat "%{User-agent}i" agent
```

```
CustomLog [% root %]/logs/access_log common
```

It goes without saying that logging formats can customized in any way. Log analyzers like consistent formats. Again, when using multiple servers it is good to configure it once and always know your web servers will stay consistent.

Templating Virtual Hosts

Template

```
[% FOREACH vhost = server.virtualhosts %]
  # For user [% vhost.user.username %]
  <VirtualHost [% vhost.ip_address %]>
    ServerName [% vhost.hostname %]
    ServerAlias www.[% vhost.hostname %] [% vhost.aliases.join(' ') %]


    DocumentRoot /home/[% vhost.user.username %]/htdocs
    ScriptAlias /cgi-bin/ "/home/[% vhost.user.username %]/cgi-bin/"


    ErrorLog    logs/[% vhost.hostname %]-error_log
    CustomLog   logs/[% vhost.hostname %]-access_log combined
  </VirtualHost>
[% END %]
```

Here is an example of configuring all the virtual hosts that belong to this server. By passing in a server object we can drill down to get all the other important information.

Program to Configure Whole Server

Update to Hosting::Server

```
package Hosting::Server;
  use base qw[Hosting::Object];
  __PACKAGE__->set_up_table( 'servers' );
  __PACKAGE__->has_many( users => 'Hosting::User' );


  __PACKAGE__->add_constructor( by_hostname => 'hostname = ?' );
  sub virtualhosts { $_->virtualhosts, $_[0]->users }
```

Here are the updates we had to do to the Hosting::Server class. Adding a constructor to retrieve a server object by the server hostname, and adding a convenience method for mapping a list of virtual hosts from the users on the server.

The Program

```
#!/usr/bin/perl
# reconfig_apache.pl
use warnings;
use strict;
```

```
use Hosting;
use Template;
use Sys::Hostname;


use constant CONF_TEMPLATE =>
      "/usr/local/apache/conf/httpd.conf.tmpl";


my $server = Hosting::Server->by_hostname( hostname );


if ( $server ) {
     my $tt = Template->new;
      $tt->process( CONF_TEMPLATE, { server => $server } )
        || die $tt->error;
} else {
      die sprintf "No configuration for %s!\n", hostname;
}
```

In this example we are going to store the httpd.conf template in a file. Instead of retrieving a virtual host object, we'll get a server object based on our hostname. If the database has information about our server we'll use the Template Toolkit to print out the configuration after processing the template.

Possible Issues

Restarting Apache with Broken Configuration

If we mis-configured apache based on the values in the database, we'll have a huge problem. When apache is restarted it will recognize the errors and fail to start back up.

Restarting Apache with Un-changed Configuration

Restarting apache when the configuration hasn't changed is just not good house keeping.

Check for Broken Configuration

New Constants

```
use constant HTTPD       => "/usr/local/apache/bin/httpd";
use constant APACHECTL  => "/usr/local/apache/bin/apachectl";
use constant CONF        => "/usr/local/apache/conf/httpd.conf";
use constant CONF_TMP   => CONF . time();
```

These constant give us a little help later on in the program.

New Condition

```
if ( $server ) {
     my $tt = Template->new;
     $tt->process(
          CONF_TEMPLATE,
```

```
            { server => $server },
            CONF_TMP,
        ) || die $tt->error;
        if ( system( HTTPD, '-T', '-f', CONF_TMP ) ) {
            die "Configuration busted: $?\n";
        }
    }
```

In this example we process the template and save it to the temporary configuration file. After that we have apache test the configuration file for syntax errors. If any errors occur, system() returns a non-zero value and the program dies displaying the error message.

Check for Un-Changed Configuration

New Module

```
use File::Compare;
```

New Condition

```
        if ( system( HTTPD, '-T', '-f', CONF_TMP ) ) {
            die "Configuration busted: $?\n";
        } elsif ( compare( CONF, CONF_TMP ) == 0 ) {
            die "Configuration not changed!\n";
        } else {
            system( 'cp', CONF_TMP, CONF ) == 0
              || die "cp failed: $?\n";
            system( APACHECTL, 'restart' );
        }
```

We follow this by using File::Compare's compare() function to see if the current configuration file is the same as the configuration file we just generated. If they are equal the program will die and note that. If they aren't equal it's time to copy the temporary config file to the real filename and restart apache.

Pros and Cons of Script

Pros

Middle Man
Checks for Problems First

Disconnecting the configuration building process from the apache startup process can give us some peace of mind about introducing problems that make apache fail at startup.

Cons

Middle Man

Because this is an extra step in the process, it's not as dynamic as some people prefer, or possibly require.

Requires Outside Help

Going to the system is often seen as ugly, and this program does it a lot.

Enter mod_perl

Dynamic Configuration

Using mod_perl to configure Apache is the epitome of dynamic configuration. How elite of us.

Cut the Middle Man

Removing a process can often be a good thing. One more process could mean one more bug.

Perl Sections

Installation Step

```
perl Makefile.PL PERL_SECTIONS=1

perl Makefile.PL EVERYTHING=1
```

If you don't compile mod_perl with one of these options, most of the rest of this talk is not helpful to you. Most binary distributions set EVERYTHING=1.

Perl Sections

Example Usage

```
<Perl>
  $Port            = 80;
   ($User, $Group)  = ('nobody')x2;
   $ServerAdmin      = 'casey@geeknest.com';
   $ServerName       = 'doppio.caseywest.com';
   $ServerSignature = 'On';
</Perl>
```

Above we're using Perl to set values in our Apache configuration. It looks quite a lot like a plain, hand written Apache configuration file. Amazing, isn't it?

Scalars, Arrays, Hashes, oh My!

Scalars

Single-value Directives

```
$Port = $< == 1 ? 80 : 8080;
```

Scalars are used to set values in an Apache configuration for directives that only ever accept one value.

Arrays

Multi-value Directives

```
push @ScriptAlias, qw[
     /cgi-bin/        /usr/local/apache/cgi-bin/
     /cgi-bin-sys/   /usr/local/sys/
];
```

Arrays are used to set directives that have multiple values. It's important in these situations to use push() or unshift() when dealing with these variables or you may clobber previous settings.

Hashes

Containers

```
$Location{'/images'} = {
     SetHandler => 'default';
};
```

Hashes are used for block constructs. The hash key is the name given to the block. If it is possible to have multiple blocks with the same name, a list reference is used for that key's value. It will be a list of hash references.

How It Works

Apache Reads Apache::ReadConfig's Symbol Table

This sounds difficult because it is. Even so, it's true. Any package variables in this namespace are used for configuring Apache. This is the default namespace assigned to sections in the configuration file.

my() Variables Ignored

Because package variables are used, lexicals are ignored. This is good, it give you a place to define variables required to build a configuration.

Configuration Variables Case-Insensitive

Just as Apache directives are case-insensitive, so too are your Perl variables. $User (with a capital 'U') is the same as $user (lower-case 'u').

Solving the VirtualHost Problem

Directive

```
<Perl>
  use Hosting;
  use Sys::Hostname;
  my $server = Hosting::Server->by_hostname( hostname );
```

```
    foreach my $vhost ( $server->virtualhosts ) {
        my ($host, $username)
              = ($vhost->hostname, $vhost->user->username);
         $VirtualHost{$vhost->ip_address} = {
            ServerName     => $host
            ServerAlias    => [
                                 'www.' . $host,
                                 $vhost->aliases,
                              ],
             DocumentRoot   => sprintf("/home/%s/htdocs", $user),
            ScriptAlias    => [
                                 '/cgi-bin/',
                                   sprintf("/home/%s/cgi-bin", $user),
                              ],
             ErrorLog       => sprintf("logs/%s-error_log",  $host),
             CustomLog      => sprintf("logs/%s-access_log", $host),
        };
    }
</Perl>
```

This bit of Perl code once again sets up this server's virtual hosts. No Template Toolkit required, we simply pull the server configuration from the database and populate the data structure.

Reviewing Our Work

With Apache::PerlSections

```
<Perl>
  ...
   warn  Apache::PerlSections->dump;
</Perl>
```

Apache::PerlSections, distributed with mod_perl, provides a nice hook to see what we've done. An Apache::ReadConfig package is output from he dump() command. Interestingly, we could also call the store() command with a filename as the argument to store that package in a file for later use.

Output

```
 package  Apache::ReadConfig;
#hashes:
...


#arrays:
...


#scalars:
...
```

dump() categorizes the directives you've defined as hash, array, and scalar. The only code you'll have in this output are variable assignments, because that's all Apache finds important in your sections.

Avoid Syntax Errors

Turn Section into Script

```
<Perl>
#!perl
...
__END__
</Perl>
```

It's important that both the hashbang and __END__ tokens begin at column zero on the line. This is a nifty trick to get Perl to syntax check for you.

Check Twice

```
perl -cx httpd.conf
```

The -c option performs a syntax check, and the -x option removes anything from the file before a hashbang (like lots of apache configuration).

Avoid Syntax Errors

Put Perl in a Script

```
PerlRequire /usr/local/apache/bin/config_vhosts.pl
```

Another method is to put your Perl configuration into a script. The name of this scrip suggests we're goign to dynamically create an apache configuration with it, when compiled by PerlRequire. We could also use PerlRequire to compile a configuration saved with Apache::PerlSections->store().

Check Twice

```
perl -c config_vhosts.pl
```

Again, syntax checking our Perl code.

What About Templates?

Apache->httpd_conf()

```
<Perl>
  use Hosting;
  use Sys::Hostname;
  my $server = Hosting::Server->by_hostname( hostname );


  foreach my $vhost ( $server->virtualhosts ) {
      my ($ip, $host, $user, $aliases) = (
          $vhost->ip_address,
          $vhost->hostname,
```

```
            $vhost->user->username,
            join( ' ', $vhost->aliases ),
        );
          Apache->httpd_conf(<<__VHOST_CONF__);
             <VirtualHost $ip>
               ServerName     $host
                ServerAliases www.$host $aliases
                DocumentRoot   /home/$user/htdocs
                ScriptAlias    /cgi-bin/ /home/$user/cgi-bin/
               ErrorLog       logs/$host-error_log
                CustomLog      logs/$host-access_log
             </VirtualHost>
  __VHOST_CONF__
  }
</Perl>
```

Apache->httpd_conf() is described as a "cheat" by some, but I find it elegant at times. Here we've gone back to templating our virtual host configuration. The template is in-line in this case and resembles our earlier Template Toolkit template.

Updating reconfig_apache.pl

Use Apache

```
my $config;
$tt->process(
     CONF_TEMPLATE,
     { server => $server },
     \$config,
) || die $tt->error;


Apache->httpd_conf( $config );
```

Using It

```
PerlRequire  /usr/local/apache/bin/reconfig_apache.pl
```

By fixing reconfig_apache.pl to use this "cheat" we can PerlRequire this script and have it build an Apache configuration at startup. In this case we're storing the output of processing the template into a scalar, $config. Then we're all set, just pass that scalar to Apache->httpd_conf().

This presenation has ended.
Click anywhere to return.

_

